

# FinPal

AI Personal Finance Manager + Grounded Chatbot

Natural Language Processing — Semester Project Report

---

8<sup>th</sup> Semester  
FAST National University

---

**Submitted By:**

Muhammad Abdullah Iqbal (22i-2504)

Nawal Hassan (22i-2428)

**Source Code Repository:**

<https://github.com/MuhammadAbdullahIqbal23/finance-ai-NLP-Project.git>

Date: May 11, 2026

## Contents

|          |                                                        |          |
|----------|--------------------------------------------------------|----------|
| <b>1</b> | <b>Project Overview</b>                                | <b>2</b> |
| <b>2</b> | <b>Features</b>                                        | <b>2</b> |
| <b>3</b> | <b>Stack We Are Using</b>                              | <b>4</b> |
| <b>4</b> | <b>Model We Are Using and Why</b>                      | <b>5</b> |
| 4.1      | Primary LLM: Llama 3.3 70B Versatile (Groq) . . . . .  | 5        |
| 4.2      | Embedding Model: Xenova/all-MiniLM-L6-v2 . . . . .     | 5        |
| <b>5</b> | <b>How We Are Using NLP in the Project</b>             | <b>5</b> |
| 5.1      | Intent and Tool Selection (function calling) . . . . . | 5        |
| 5.2      | Multi-step Reasoning over Tools . . . . .              | 6        |
| 5.3      | Semantic Retrieval (RAG over Transactions) . . . . .   | 6        |
| 5.4      | Output Generation and Grounding Validation . . . . .   | 6        |
| <b>6</b> | <b>Sequence Diagram</b>                                | <b>6</b> |
| <b>7</b> | <b>Architecture Diagram</b>                            | <b>7</b> |
| <b>8</b> | <b>Completion Criteria (What We Have Accomplished)</b> | <b>7</b> |
| <b>9</b> | <b>Conclusion</b>                                      | <b>9</b> |

## 1. Project Overview

FinPal is an AI personal finance manager for a single Pakistani user. It tracks income, expenses, and savings over three months, forecasts next-month spend per category, and answers questions through a **tool-calling chatbot**. Core rule: every PKR figure the bot cites comes from a real server-side function call — never invented by the model.

Built as a Next.js 14 monolith. Server Components query SQLite via Prisma. The chat route orchestrates Groq-hosted Llama 3.3 70B via Vercel AI SDK’s `streamText`, exposing 11 typed tools. Local MiniLM embeddings power semantic retrieval over transactions for fuzzy queries (e.g. “the big Daraz purchase”).

Design goal: honesty under data scarcity. With  $n = 3$  months (PKR 293,107,  $\sim 128$  transactions), the system uses EWMA + linear trend instead of deep learning, and a four-layer anti-hallucination pipeline keeps every user-visible number auditable.

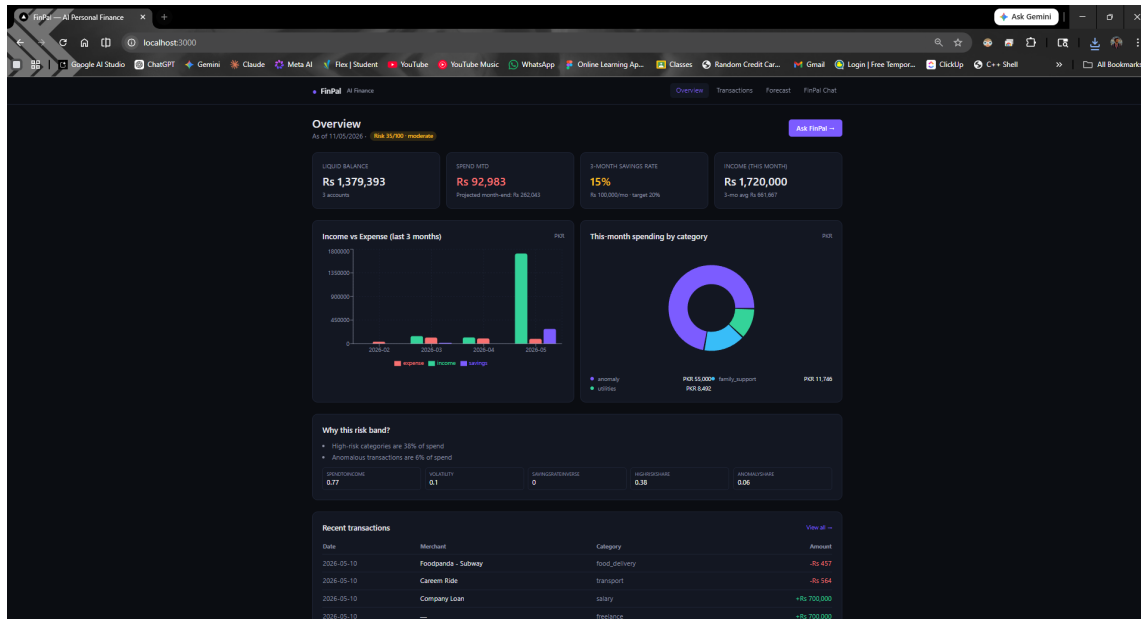


Figure 1: FinPal Overview Dashboard — KPIs, monthly trend, category breakdown, recent transactions

## 2. Features

- **Synthetic Pakistani spending dataset** — 3 months (Feb 10 – May 10, 2026),  $\sim 128$  transactions across categories: Food, Groceries, Transport, Utilities, Shopping, Bills, Entertainment. Real local merchants: Foodpanda, Cheezious, K-Electric, JazzCash, Daraz, FAST University. Includes day-of-week effects, salary cycles, one missed savings transfer, and two seeded anomalies.
- **Forecast engine** — EWMA ( $\alpha = 0.5$ ) blended with linear trend across three monthly aggregates per category. Confidence band shown. Honest about the  $n = 3$  statistical limit.
- **Rule engine** — affordability check, installment safety check (EMI / disposable income ratio), 5-component risk score (0–100), monthly health check, savings recommendation, savings-goal projection.
- **Local RAG pipeline** — per-transaction embeddings via Transformers.js (Xenova/all-MiniLM-L6-v2, 384 dimensions). Cosine top- $k$  retrieval with recency boost. Runs in Node.js, no API key needed.

- **Tool-calling chatbot** — Vercel AI SDK + Groq, 11 tools, multi-step reasoning ( $\text{maxSteps}=6$ ), Live Context Block in the system prompt, output validator that flags ungrounded numbers.
- **Dashboard** — Overview KPIs (liquid balance, MTD spend with projected month-end, savings rate, income), monthly trend chart, category pie, risk breakdown panel, recent transactions table, paginated browser, forecast chart, affordability simulator, savings goal tracker.
- **Anti-hallucination pipeline** — 4 layers: system prompt rule, Live Context Block (authoritative numbers in every prompt), tool-only number policy, output validator (parses every PKR / % in the answer and confirms it exists within  $\pm 2\%$  tolerance in tool results).

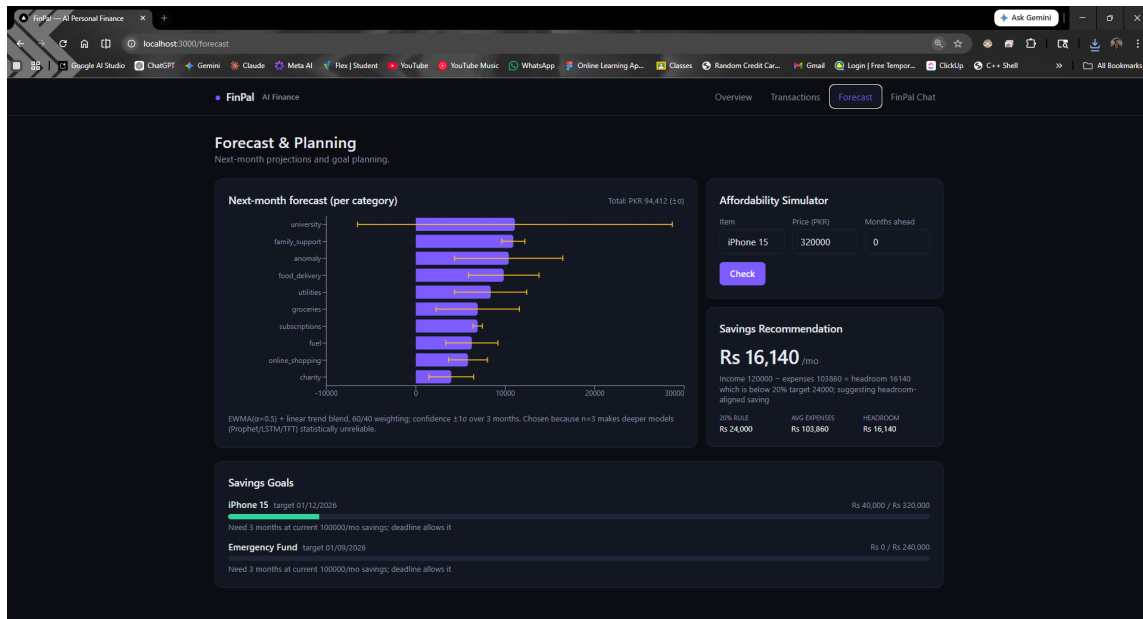


Figure 2: Forecast Page with Affordability Simulator

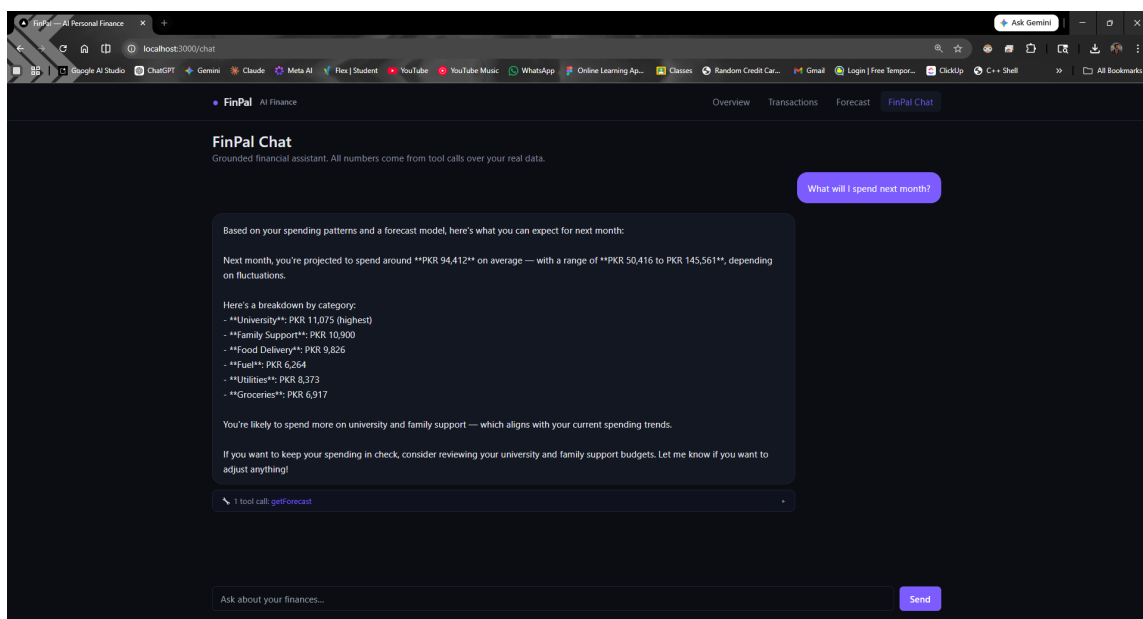


Figure 3: Chatbot with Expandable Tool-Call Trace

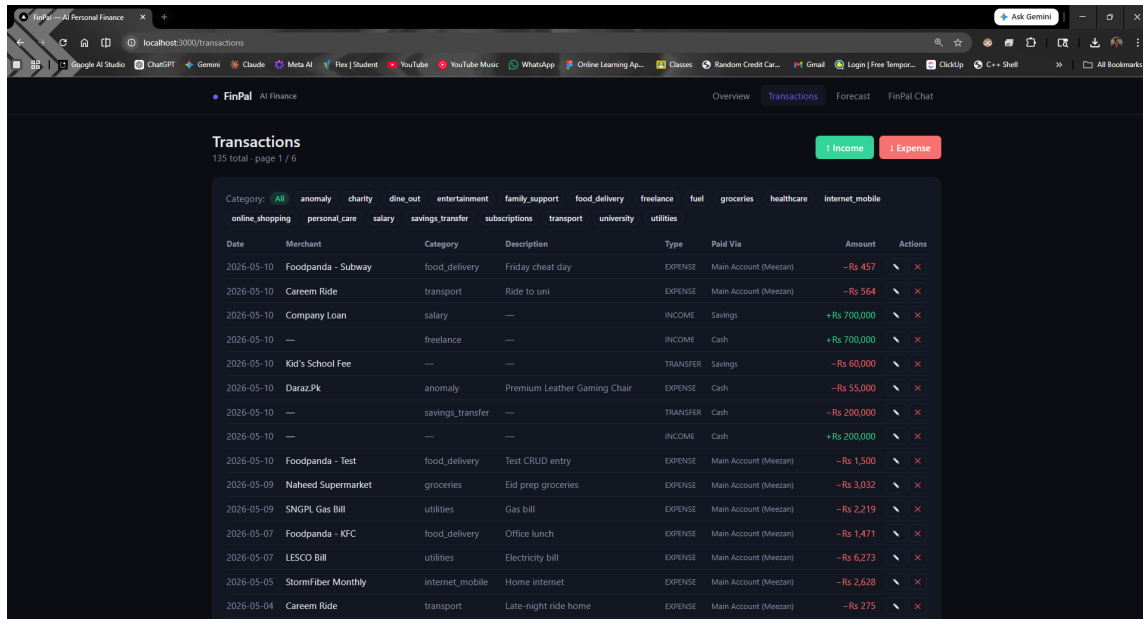


Figure 4: Transactions Browser — paginated view of all synthetic transactions

### 3. Stack We Are Using

| Layer              | Technology                                                              |
|--------------------|-------------------------------------------------------------------------|
| Frontend Framework | Next.js 14 (App Router) + React 18                                      |
| Language           | TypeScript                                                              |
| Styling            | Tailwind CSS                                                            |
| Charts             | Recharts                                                                |
| ORM                | Prisma 6                                                                |
| Database           | SQLite (file-based, embedded)                                           |
| LLM Orchestration  | Vercel AI SDK 4 ( <code>streamText</code> ) + <code>@ai-sdk/groq</code> |
| Language Model     | Llama 3.3 70B Versatile (hosted on Groq)                                |
| Embedding Model    | Xenova/all-MiniLM-L6-v2 (384-dim, runs locally via Transformers.js)     |
| Data Generation    | Faker.js (Pakistani-localized synthetic data)                           |
| Runtime            | Node.js 20+                                                             |

Table 1: Technology Stack

#### Why this stack:

- **Next.js monolith** — Server Components fetch from the database directly; no separate backend. Single repo, single deploy.
- **SQLite + Prisma** — zero-budget, file-based, fast enough for a single-user demo. Migrations are reproducible.
- **Groq inference** — free tier with very fast token streaming, no GPU required locally.

- **Transformers.js** — runs the embedding model in pure JavaScript, eliminating Python dependency and external vector-DB cost.

## 4. Model We Are Using and Why

### 4.1. Primary LLM: Llama 3.3 70B Versatile (Groq)

**Choice:** Meta's Llama 3.3 70B Versatile, served via the Groq inference platform.

**Why:**

- **Strong tool-calling.** Llama 3.3 70B was tuned for structured function calling. The chatbot depends on the model emitting valid JSON tool calls against an 11-tool schema — weaker models hallucinate tool names or invalid arguments.
- **Latency.** Groq's LPU inference delivers  $\sim 300+$  tokens/sec. Streaming feels real-time, which matters for a viva-time demo.
- **Cost.** Free tier accommodates project scale.
- **Reasoning depth.** 70B parameters handle multi-step tool reasoning (`maxSteps=6`) — the chatbot can chain `getCategoryBreakdown`  $\rightarrow$  `getRiskScore` in a single turn without losing context.
- **Open-weight model.** No vendor lock-in to a closed model; can be swapped to a self-hosted Llama deployment in production.

### 4.2. Embedding Model: Xenova/all-MiniLM-L6-v2

**Choice:** a 6-layer sentence-transformer producing 384-dimensional embeddings, run locally in Node via Transformers.js.

**Why:**

- **Tiny footprint** ( $\sim 25$  MB) downloaded once on first run, cached forever.
- **No API key, no Python.** Critical for a single-deploy demo.
- **Latency.** Encoding a query and computing cosine similarity over  $\sim 128$  transactions takes milliseconds in-memory.
- **Quality / cost trade-off.** For semantic search over short transaction descriptions (e.g. "Daraz iPhone 15 purchase"), MiniLM-L6 is empirically sufficient; spending API budget on `text-embedding-3-large` adds no measurable benefit at this corpus size.

## 5. How We Are Using NLP in the Project

NLP drives every intelligent feature in FinPal across **four layers**:

### 5.1. Intent and Tool Selection (function calling)

For a query like "*Can I afford an iPhone in 6 months?*", Llama 3.3 70B:

1. Parses intent (affordability check),
2. Extracts slots (item, horizon, price),
3. Emits a typed tool call (`runAffordability({itemName, priceInPKR, monthsAhead})`).

Classical NLP *slot filling* delegated to an LLM.

## 5.2. Multi-step Reasoning over Tools

With `maxSteps=6`, the model chains calls — e.g. `getCategoryBreakdown`  $\rightarrow$  `getRiskScore` — using the first result to decide the second. Contextual reasoning over structured data.

## 5.3. Semantic Retrieval (RAG over Transactions)

Fuzzy queries (“*the big Daraz purchase*”) flow through:

1. Encode query with MiniLM-L6-v2 (384-dim).
2. Cosine similarity against pre-computed transaction embeddings.
3. Recency boost.
4. Top- $k$  transactions returned as grounded context.

Standard RAG, fully in-process.

## 5.4. Output Generation and Grounding Validation

Model writes the final answer citing tool-sourced figures. A regex validator then parses every `PKR / %` span and checks it against tool results or the Live Context Block ( $\pm 2\%$ ). Information-extraction NLP applied to the model’s own output.

| NLP Task                             | Where It Appears in FinPal                               |
|--------------------------------------|----------------------------------------------------------|
| Intent classification & slot filling | User-message $\rightarrow$ tool-call mapping             |
| Multi-step reasoning                 | Chained tool calls ( <code>maxSteps=6</code> )           |
| Dense retrieval (semantic search)    | Transaction RAG via MiniLM embeddings                    |
| Natural-language generation          | Final chatbot answer                                     |
| Numeric entity extraction            | Output validator scanning for <code>PKR / %</code> spans |
| Contextual grounding                 | Live Context Block injected each turn                    |

Table 2: NLP Tasks Realized in FinPal

## 6. Sequence Diagram

The sequence below describes a typical user query through the chatbot, from message submission to grounded response.

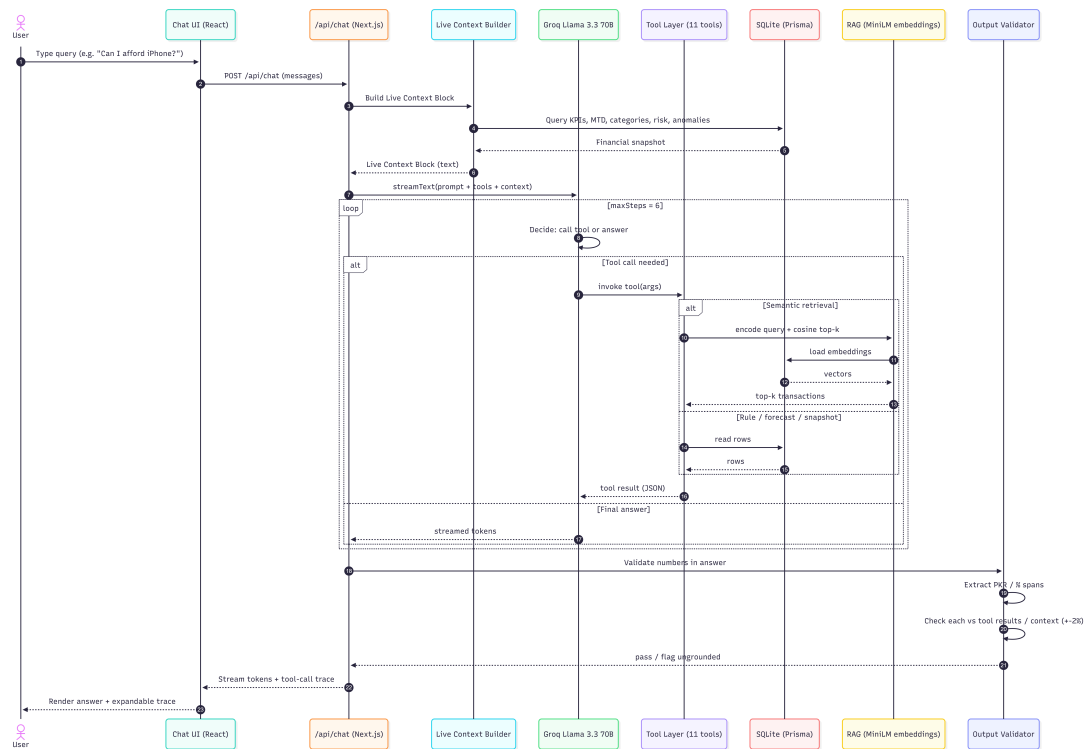


Figure 5: Chatbot Request Sequence Diagram

## 7. Architecture Diagram

The architecture diagram shows the major components of FinPal and how data flows between them: from the Next.js UI, through the API routes, into the chatbot orchestrator (LLM + tools + RAG), and out to the SQLite store.

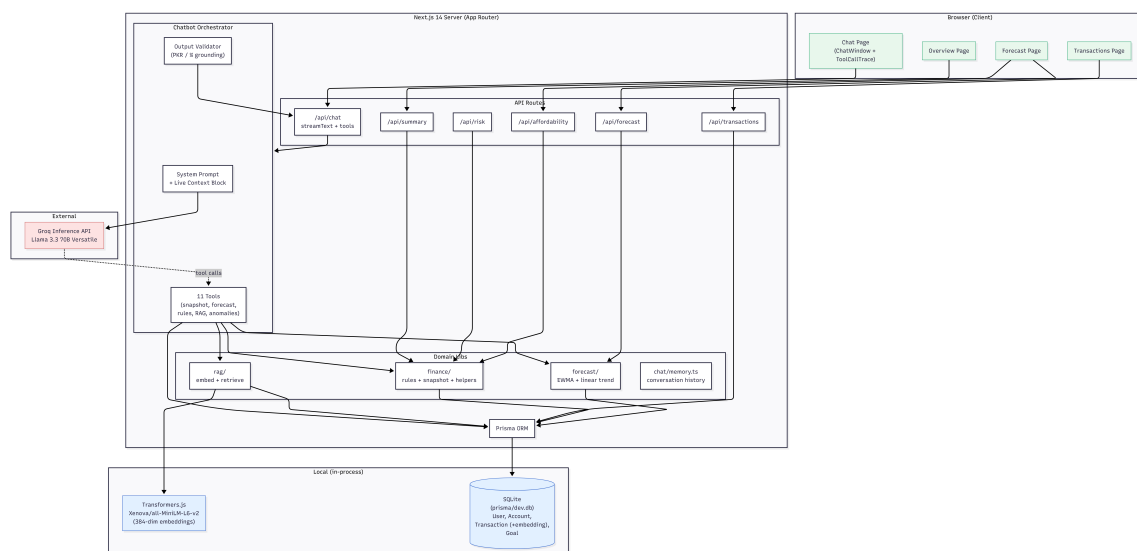


Figure 6: FinPal System Architecture

## 8. Completion Criteria (What We Have Accomplished)

The following items represent the completed scope of the project as of the date of this report:



- [Done] Next.js 14 App Router project scaffolded with TypeScript, Tailwind, and Prisma.
- [Done] SQLite schema designed with tables for User, Account, Transaction, Goal, and a **Bytes** column for transaction embeddings.
- [Done] Synthetic data seeder generates 3 months of realistic Pakistani transactions (~128 rows, total in PKR 200k–300k range, asserted by seeder).
- [Done] EWMA + linear-trend forecast engine with confidence band.
- [Done] Rule engine: affordability, installment safety, 5-component risk score, monthly health, savings recommendation, goal projection.
- [Done] RAG pipeline: precompute embeddings, runtime cosine top- $k$  retrieval with recency boost.
- [Done] 11 typed tools exposed to the Groq Llama 3.3 70B model via Vercel AI SDK.
- [Done] Chatbot endpoint (`/api/chat`) with multi-step tool reasoning, streaming responses.
- [Done] Live Context Block injected into the system prompt every turn.
- [Done] Output validator parsing PKR / % spans and verifying against tool results.
- [Done] Dashboard UI: Overview KPIs, monthly trend chart, category pie, risk breakdown, recent transactions, paginated browser.
- [Done] Forecast page with affordability simulator.
- [Done] Chat page with sample-question chips and an expandable tool-call trace panel for viva demonstration.
- [Done] Documentation: README + architecture document + this report.

**Out-of-scope but acknowledged:** multi-user authentication (scaffolded, not wired), CSV import for real Pakistani bank statements (HBL, Meezan, JazzCash), deep-learning forecasters (Prophet, LSTM, TFT) — intentionally avoided due to  $n = 3$  monthly aggregates.

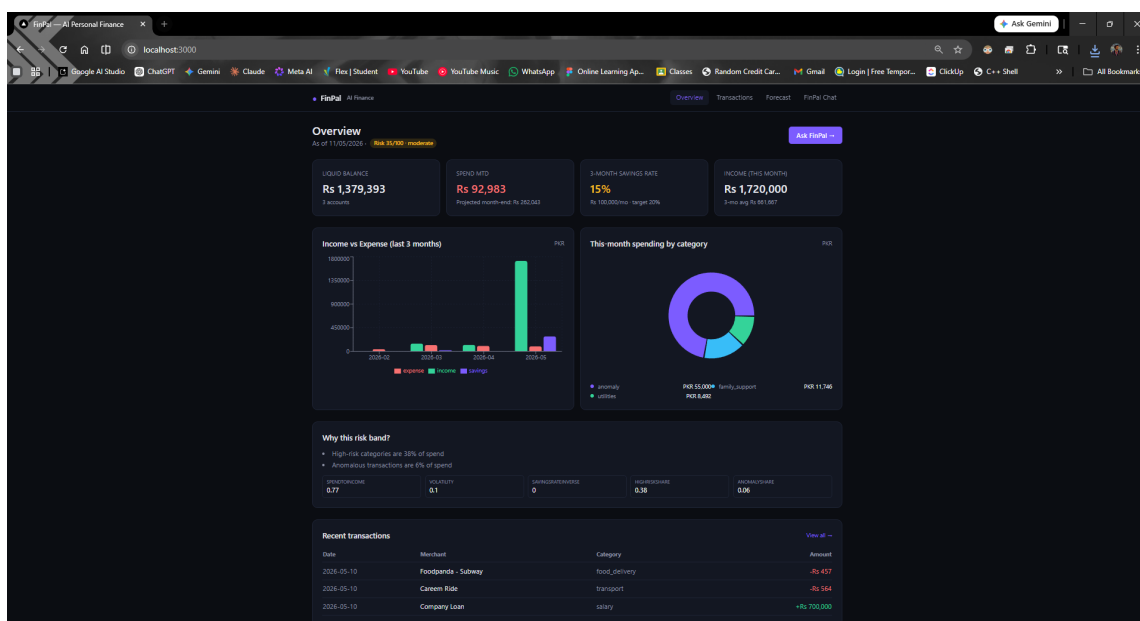


Figure 7: Final Application View — Overview Dashboard

## 9. Conclusion

FinPal shows a production-flavoured NLP app can be built end-to-end on a zero-budget stack while staying honest about its limits. Core bet: **numbers from tools, words from the model** — every figure is auditable through a visible tool-call trace.

Four NLP capabilities (function-call intent parsing, multi-step tool reasoning, dense retrieval, output grounding) sit inside one Next.js monolith. Only LLM inference is remote (Groq free tier). The validator surfaces ungrounded numbers instead of silently passing them.

Key takeaway: for small, structured domains ( $n = 3$  months, 7 categories), pick the simplest statistical method that fits the data and the strongest LLM available for tool orchestration — not the deepest model.